

LLD-07: Concurrency-2.5 — Executor Syntax, Parallel Merge Sort, and Mutex

Last class you learned WHAT thread pools and futures are. This class, you learn HOW to use them — every method, every pattern. Then we break things with shared state and fix them with locks.

1. Quick Recap

No peeking at last class's notes. Answer from memory.

Q1: What is a thread pool? Draw it in your head.

► Answer

Q2: What is a Future? (Think receipt.)

► Answer

Q3: ThreadPoolExecutor for ___-bound work. ProcessPoolExecutor for ___-bound work. Why?

► Answer

Q4: What's the `with` statement doing in `with ThreadPoolExecutor() as executor: ?`

► Answer

2. executor.submit() — Hands On

Last class showed `submit()` conceptually. Now let's break down every line.

The Syntax

```
from concurrent.futures import ThreadPoolExecutor
import time

def fetch_data(url, retries=3):
    """Simulates an API call."""
    print(f" Fetching {url} (retries={retries})...")
    time.sleep(2)
    return f"Data from {url}"

with ThreadPoolExecutor(max_workers=3) as executor:
    # Submit a single task to the pool
    future = executor.submit(fetch_data, "https://api.razorpay.com/orders", retries=2)
    # ~~~~~
    # the executor function arguments (positional AND keyword)
    # to run passed directly to fetch_data()
```

Line-by-line:

1. `executor.submit(func, arg1, arg2, kwarg=val)` — submit ONE task to the pool
2. It returns a **Future** immediately — does NOT wait for the task to finish
3. Arguments after `func` are passed directly to `func` when a worker picks it up
4. The pool assigns it to an available worker thread (or queues it if all workers are busy)

Question: What's the difference between `executor.submit(fetch_data, url)` and `executor.submit(fetch_data(url))` ?

► Answer

What submit() Returns — The Future Lifecycle

```
from concurrent.futures import ThreadPoolExecutor
import time

def slow_task(name, seconds):
    time.sleep(seconds)
    return f"{name} completed after {seconds}s"

with ThreadPoolExecutor(max_workers=2) as executor:
    future = executor.submit(slow_task, "Payment", 3)

    # Stage 1: SUBMITTED (task is queued or running)
    print(future.done())    # False – still running
    print(future.running()) # True  – a worker picked it up

    # Stage 2: WAITING for result
    result = future.result() # BLOCKS here until the task finishes
    #      ~~~~~
    #      Main thread STOPS and waits. Like .join() but for one task.

    # Stage 3: DONE
    print(future.done())    # True  – finished!
    print(result)           # "Payment completed after 3s"
```

The lifecycle of a Future:

submit()	worker picks it up	task finishes
v	v	v
PENDING	-----> RUNNING	-----> FINISHED
		(result or exception stored)

future.done() — Non-blocking Check

`.done()` returns `True` or `False` **immediately**. It never blocks.

```
future = executor.submit(slow_task, "Email", 5)
print(future.done()) # False – still working
time.sleep(6)
print(future.done()) # True  – finished
```

Use `.done()` when you want to check without waiting. Useful for progress indicators or doing other work while tasks run.

future.result() — Block Until Done

`.result()` **blocks** the calling thread until the task finishes and returns the value.

```
future = executor.submit(slow_task, "SMS", 2)

# This line BLOCKS for ~2 seconds:
result = future.result()
print(result) # "SMS completed after 2s"
```

future.result(timeout=N) — Don't Wait Forever

What if the task hangs? Set a timeout.

```
from concurrent.futures import TimeoutError

future = executor.submit(slow_task, "Razorpay", 10)

try:
    result = future.result(timeout=5) # Wait at most 5 seconds
except TimeoutError:
    print("Task took too long! Moving on.")
    # The task is STILL RUNNING in the background — timeout doesn't cancel it
```

Question: After a `TimeoutError`, is the task cancelled?

► Answer

Putting It All Together — Timed Example

```
from concurrent.futures import ThreadPoolExecutor
import time

def simulate_api(name, delay):
    """Simulates an API call with a given delay."""
    print(f" [{time.strftime('%H:%M:%S')}] {name}: starting...")
    time.sleep(delay)
    print(f" [{time.strftime('%H:%M:%S')}] {name}: done!")
    return f"{name} result"

print("=== Submitting 3 tasks ===")
start = time.time()

with ThreadPoolExecutor(max_workers=3) as executor:
    f1 = executor.submit(simulate_api, "Razorpay", 3)
    f2 = executor.submit(simulate_api, "Email", 1)
    f3 = executor.submit(simulate_api, "SMS", 0.5)

    # All 3 are running concurrently right now!

    # Check status before waiting
    print(f" f1 done? {f1.done()}") # False (3s task)
    print(f" f2 done? {f2.done()}") # False (1s task)
    print(f" f3 done? {f3.done()}") # False (0.5s task)

    # Get results (blocks until each is ready)
    print(f"\n Result: {f1.result()}") # waits ~3s
    print(f" Result: {f2.result()}") # already done — returns instantly
    print(f" Result: {f3.result()}") # already done — returns instantly

print(f"\nTotal time: {time.time() - start:.1f}s")
# Total: ~3s (not 4.5s) — all ran concurrently!
```

3. as_completed() — Process Results as They Arrive

The Problem

You submitted 3 tasks with different durations. If you call `.result()` on each in order, you wait for the slowest one first:

```
# This waits 3 seconds for f1 before printing ANYTHING
# Even though f3 finished in 0.5 seconds!
print(f1.result()) # blocks 3s
print(f2.result()) # already done, instant
print(f3.result()) # already done, instant
```

What if you want to print results as each finishes? SMS finishes in 0.5s — print it immediately. Don't wait for Razorpay's 3 seconds.

The Solution: `as_completed()`

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import time

def simulate_api(name, delay):
    time.sleep(delay)
    return f"{name} done (took {delay}s)"

start = time.time()

with ThreadPoolExecutor(max_workers=3) as executor:
    futures = {
        executor.submit(simulate_api, "Razorpay", 3): "Razorpay",
        executor.submit(simulate_api, "Email", 1): "Email",
        executor.submit(simulate_api, "SMS", 0.5): "SMS",
    }

    # as_completed() yields futures in the order they FINISH
    for future in as_completed(futures):
        name = futures[future]
        result = future.result()
        elapsed = time.time() - start
        print(f" [{elapsed:.1f}s] {result}")

# Output:
# [0.5s] SMS done (took 0.5s)      <-- fastest first!
# [1.0s] Email done (took 1s)
# [3.0s] Razorpay done (took 3s)  <-- slowest last
```

Key insight: `as_completed()` returns futures in **completion order** — fastest first. Not submission order.

Why This Matters

Imagine a dashboard that shows API health checks. You have 20 services to ping. Some respond in 50ms, some in 5 seconds. With `as_completed()`, you can update the dashboard as each result comes in — the user sees fast results immediately instead of staring at a blank screen for 5 seconds.

The Pattern — Dictionary Mapping

Notice the dictionary pattern:

```
# Map each future back to its input
futures = {executor.submit(func, arg): arg for arg in args}

for future in as_completed(futures):
    original_arg = futures[future] # look up which input this future was for
    result = future.result()
```

This is the standard pattern because `as_completed()` gives you futures, and you need to know WHICH task each future corresponds to.

Question: Can `as_completed()` be used with `executor.map()` ?

► Answer

4. executor.map() — The Simple Way

map() is Parallel map()

Python's built-in `map()` applies a function to every item in an iterable:

```
# Built-in map: sequential
results = map(str.upper, ["hello", "world", "python"])
# --> ["HELLO", "WORLD", "PYTHON"]
```

`executor.map()` does the same thing, but runs the function calls **in parallel** across the worker pool:

```
from concurrent.futures import ThreadPoolExecutor
import time

def fetch_url(url):
    print(f" Fetching {url}...")
    time.sleep(1) # simulate network delay
    return f"Data from {url}"

urls = [
    "https://api.example.com/users",
    "https://api.example.com/orders",
    "https://api.example.com/products",
    "https://api.example.com/payments",
    "https://api.example.com/notifications",
]

start = time.time()

with ThreadPoolExecutor(max_workers=5) as executor:
    # Like map(), but parallel
    results = executor.map(fetch_url, urls)
    #           ^^^^^^^^^^^  ^^^^^^^^^  ^^^^^
    #           parallel map  function   iterable of inputs

    for result in results:
        print(f" {result}")
```

```
print(f"\nTotal: {time.time() - start:.1f}s")
# ~1s total (not 5s) – all 5 fetched concurrently!
```

Key Properties of map()

1. **Returns results in SUBMISSION order** — always. If URLs are [A, B, C], results are [A_result, B_result, C_result], regardless of which finished first.
2. **Returns an iterator**, not a list — results are yielded as they become available, but in order. If A takes 5s and B takes 1s, iterating blocks at A for 5 seconds even though B is already done.
3. **Same function for all inputs** — you can't use different functions for different items.
4. **Simpler syntax** — no Futures to manage.

Multiple Iterables

Like the built-in `map()`, you can pass multiple iterables:

```
def send_notification(user, message):
    time.sleep(0.5)
    return f"Sent '{message}' to {user}"

users = ["alice", "bob", "charlie"]
messages = ["Welcome!", "Your order shipped", "Payment received"]

with ThreadPoolExecutor(max_workers=3) as executor:
    results = executor.map(send_notification, users, messages)
    # Calls: send_notification("alice", "Welcome!")
    #         send_notification("bob", "Your order shipped")
    #         send_notification("charlie", "Payment received")

    for result in results:
        print(f" {result}")
```

Setting a Timeout

```
with ThreadPoolExecutor(max_workers=5) as executor:
    # If ANY task takes longer than 10 seconds, raise TimeoutError
    results = executor.map(fetch_url, urls, timeout=10)
```

5. map() vs submit() — When to Use Which

	<code>executor.map()</code>	<code>executor.submit()</code>
Returns	Iterator of results	Future objects
Result order	Submission order (always)	Completion order (with <code>as_completed()</code>)
Flexibility	Same function for all inputs	Different functions, different args
Error handling	Exception on iteration (hard to isolate)	Per-task try/except on <code>.result()</code>
Syntax	Simple — one line	More code, more control
Best for	"Apply this function to all these inputs"	"Run these different tasks, handle each result"

Same Problem, Both Ways

Problem: Fetch 3 URLs with different simulated delays.

With map() — simple, ordered:

```
from concurrent.futures import ThreadPoolExecutor
import time

def fetch(url):
    delay = {"users": 3, "orders": 1, "payments": 0.5}
    name = url.split("/")[-1]
    time.sleep(delay.get(name, 1))
    return f"{name} done"

urls = ["https://api.example.com/users",
        "https://api.example.com/orders",
        "https://api.example.com/payments"]

with ThreadPoolExecutor(max_workers=3) as executor:
    for result in executor.map(fetch, urls):
        print(result)

# Output (always this order, even though payments finished first):
# users done      <-- waited 3s for this before printing anything
# orders done
# payments done
```

With submit() + as_completed() — flexible, fastest-first:

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import time

def fetch(url):
    delay = {"users": 3, "orders": 1, "payments": 0.5}
    name = url.split("/")[-1]
    time.sleep(delay.get(name, 1))
    return f"{name} done"

urls = ["https://api.example.com/users",
        "https://api.example.com/orders",
        "https://api.example.com/payments"]

with ThreadPoolExecutor(max_workers=3) as executor:
    futures = {executor.submit(fetch, url): url for url in urls}
    for future in as_completed(futures):
        print(future.result())

# Output (fastest first):
# payments done  <-- 0.5s
# orders done   <-- 1s
# users done    <-- 3s
```

Question: You need to call `verify_payment(order_id)`, `send_email(user)`, and `send_sms(phone)` concurrently. These are three DIFFERENT functions. Would you use `map()` or `submit()` ?

► Answer

6. Parallel Merge Sort — A Real Algorithm

Time to use executors on a real algorithm, not just simulated delays.

First: What is Merge Sort?

Merge sort is a divide-and-conquer sorting algorithm:

1. **Split** the list in half
2. **Sort** each half (recursively)
3. **Merge** the two sorted halves

```
Original:      [38, 27, 43, 3, 9, 82, 10]

Split:         [38, 27, 43, 3]      [9, 82, 10]

Split again:   [38, 27] [43, 3]     [9, 82] [10]

Split again:   [38] [27] [43] [3]   [9] [82] [10]

Merge:         [27, 38] [3, 43]     [9, 82] [10]

Merge:         [3, 27, 38, 43]      [9, 10, 82]

Merge:         [3, 9, 10, 27, 38, 43, 82]
```

Sequential Merge Sort

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])    # Sort left half
    right = merge_sort(arr[mid:])    # Sort right half
    return merge(left, right)       # Merge sorted halves

def merge(left, right):
    """Merge two sorted lists into one sorted list."""
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Can We Parallelize It?

Look at the recursive step:

```
left = merge_sort(arr[:mid])    # Sort left half
right = merge_sort(arr[mid:])   # Sort right half
```

These two calls are **independent** — sorting the left half doesn't depend on the right half. They can run in parallel!

But the merge step:

```
return merge(left, right)      # MUST wait for both halves
```

This is sequential — you need both sorted halves before merging. This is a textbook example of **Amdahl's Law** — only part of the work can be parallelized.

ThreadPoolExecutor Attempt (Spoiler: No Speedup)

```
from concurrent.futures import ThreadPoolExecutor
import time, random

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def parallel_merge_sort_threads(arr, executor, depth=0):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    if depth < 2: # Only parallelize the top levels
        future_left = executor.submit(parallel_merge_sort_threads, arr[:mid], executor, depth + 1)
        future_right = executor.submit(parallel_merge_sort_threads, arr[mid:], executor, depth + 1)
        left = future_left.result()
        right = future_right.result()
    else:
        left = parallel_merge_sort_threads(arr[:mid], executor, depth + 1)
        right = parallel_merge_sort_threads(arr[mid:], executor, depth + 1)

    return merge(left, right)

# Test
data = [random.randint(0, 1_000_000) for _ in range(500_000)]

start = time.time()
sorted_seq = merge_sort(data[:])
print(f"Sequential: {time.time() - start:.2f}s")
```

```

start = time.time()
with ThreadPoolExecutor(max_workers=4) as executor:
    sorted_par = parallel_merge_sort_threads(data[:], executor)
print(f"Threaded:    {time.time() - start:.2f}s")

# Sequential:  ~1.2s
# Threaded:    ~1.3s  (SAME or SLOWER!)

```

Why no speedup? Merge sort is **CPU-bound** — it's pure computation (comparisons, list operations). The GIL means only one thread runs Python at a time. You're paying thread overhead for zero parallelism.

ProcessPoolExecutor — Real Speedup

```

from concurrent.futures import ProcessPoolExecutor
import time, random

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def sort_chunk(chunk):
    """Sort a chunk — this runs in a separate process."""
    return merge_sort(chunk)

def parallel_merge_sort_processes(arr, num_workers=4):
    """Split into chunks, sort each in a process, merge results."""
    chunk_size = len(arr) // num_workers
    chunks = [arr[i * chunk_size:(i + 1) * chunk_size] for i in range(num_workers)]
    # Handle remainder
    if len(arr) % num_workers:
        chunks[-1].extend(arr[num_workers * chunk_size:])

    with ProcessPoolExecutor(max_workers=num_workers) as executor:
        sorted_chunks = list(executor.map(sort_chunk, chunks))

    # Merge all sorted chunks
    result = sorted_chunks[0]

```

```

    for chunk in sorted_chunks[1:]:
        result = merge(result, chunk)
    return result

if __name__ == "__main__":
    data = [random.randint(0, 1_000_000) for _ in range(500_000)]

    start = time.time()
    sorted_seq = merge_sort(data[:])
    print(f"Sequential: {time.time() - start:.2f}s")

    start = time.time()
    sorted_par = parallel_merge_sort_processes(data[:], num_workers=4)
    print(f"Processes: {time.time() - start:.2f}s")

    # Verify correctness
    assert sorted_seq == sorted_par
    print("Results match!")

    # Sequential: ~1.2s
    # Processes: ~0.5s (REAL speedup!)

```

Why processes work: Each process has its own GIL and runs on a separate core. True parallelism for CPU-bound sorting.

Question: The speedup is ~2.4x with 4 processes, not 4x. Why isn't it a perfect 4x speedup?

► Answer

7. The Race Condition Problem — Why We Need Locks

Back to threads and shared memory. This is where things get dangerous.

The Classic Bug

```

import threading

counter = 0

def increment():
    global counter
    for _ in range(100_000):
        counter += 1

# Two threads, each incrementing 100,000 times
t1 = threading.Thread(target=increment)
t2 = threading.Thread(target=increment)

t1.start()
t2.start()
t1.join()
t2.join()

print(f"Expected: 200,000")
print(f"Actual: {counter}")

```

Run this. You expect 200,000. You'll get something like **187,432**. Run it again: **192,017**. Again: **189,556**. **Different every time.**

WHY: counter += 1 is NOT Atomic

`counter += 1` **looks** like one operation. It's actually three:

```
Step 1: READ   counter from memory into a register
Step 2: ADD    1 to the register value
Step 3: WRITE  the new value back to memory
```

A context switch can happen between ANY of these steps.

The Interleaving That Breaks Things

counter starts at 5.

```
Thread A: READ counter = 5      (A has 5 in its register)
                                     --- context switch! ---
Thread B: READ counter = 5      (B ALSO reads 5 – same stale value!)
Thread B: ADD 1 --> 6
Thread B: WRITE counter = 6     (counter is now 6)
                                     --- context switch back ---
Thread A: ADD 1 --> 6           (A still has 5, adds 1 = 6)
Thread A: WRITE counter = 6     (counter is STILL 6, not 7!)
```

Two increments happened. Counter went from 5 to 6 instead of 5 to 7.
Thread A's work was LOST – overwritten by the stale value.

This is a **race condition** — the result depends on the unpredictable timing of thread execution. It's non-deterministic: sometimes you get the right answer, sometimes you don't.

Question: *Would reducing the loop from 100,000 to 10 fix the race condition?*

► Answer

Question: *If `counter += 1` is three steps, is there ANY Python operation that IS atomic (can't be interrupted)?*

► Answer

8. Mutex — The Simplest Lock

Mutex = Mutual Exclusion

A **mutex** (mutual exclusion lock) is the simplest synchronization tool. The rule:

Only ONE thread can hold the lock at any time. All others wait.

Think of it as a bathroom with a lock:

- Person enters, **locks** the door
- Other people arrive, see it's locked, **wait in line**
- Person finishes, **unlocks** the door
- Next person in line enters and locks it

threading.Lock()

```
import threading

lock = threading.Lock()
```

```
# Method 1: Manual acquire/release
lock.acquire()      # Grab the lock (blocks if someone else has it)
# ... do work ...
lock.release()      # Release the lock (next waiting thread can grab it)

# Method 2: Context manager (MUCH better – release is guaranteed)
with lock:
    # ... do work ...
    # lock is automatically released when this block exits,
    # even if an exception occurs
```

Always use `with lock:` . If you use `acquire()` / `release()` manually and an exception happens between them, the lock is never released. Every other thread waits forever. Deadlock.

Fixing the Counter with a Lock

```
import threading

counter = 0
lock = threading.Lock()

def increment():
    global counter
    for _ in range(100_000):
        with lock:
            counter += 1      # Only ONE thread can be in here at a time
                             # READ, ADD, WRITE – all protected

t1 = threading.Thread(target=increment)
t2 = threading.Thread(target=increment)
t1.start()
t2.start()
t1.join()
t2.join()

print(f"Expected: 200,000")
print(f"Actual:    {counter}")
# Actual: 200,000 -- ALWAYS. Every time. Guaranteed.
```

What the Lock Does

```
WITHOUT lock:
Thread A: READ(5) -----> ADD(6) -> WRITE(6)
Thread B:          READ(5) -----> ADD(6) -> WRITE(6)
Result: 6 (should be 7)

WITH lock:
Thread A: [LOCK] READ(5) -> ADD(6) -> WRITE(6) [UNLOCK]
Thread B:          (waiting...)          [LOCK] READ(6) -> ADD(7) -> WRITE(7) [UNLOCK]
Result: 7 (correct!)
```

The lock ensures the READ-ADD-WRITE sequence is **atomic** — it can't be interrupted by another thread accessing the same data.

The Performance Cost

Locking is not free. Let's measure:

```

import threading, time

counter = 0
lock = threading.Lock()

def increment_no_lock():
    global counter
    for _ in range(1_000_000):
        counter += 1

def increment_with_lock():
    global counter
    for _ in range(1_000_000):
        with lock:
            counter += 1

# Without lock (single thread – no race condition)
counter = 0
start = time.time()
increment_no_lock()
print(f"No lock:    {time.time() - start:.3f}s  counter={counter}")

# With lock (single thread – shows pure lock overhead)
counter = 0
start = time.time()
increment_with_lock()
print(f"With lock:  {time.time() - start:.3f}s  counter={counter}")

# No lock:    ~0.06s   counter=1000000
# With lock:  ~0.25s   counter=1000000
# Lock adds ~4x overhead! acquire() and release() 1,000,000 times.

```

Locks are necessary for correctness but expensive. This is why you should lock the **minimum necessary** — don't wrap your entire function in a lock.

Question: *If locks slow things down so much, why not just use a single thread and avoid locks entirely?*

► Answer

Lock Granularity — Lock Less, Not More

Bad — locking too much:

```

def process_order(order):
    with lock:
        validate(order)
        payment = call_razorpay(order)
        update_database(order)
        send_email(order)

```

Locked the ENTIRE function!
 # This doesn't touch shared data
 # 500ms API call — all threads wait!
 # This touches shared data
 # This doesn't need the lock

Good — locking only what needs it:

```

def process_order(order):
    validate(order)
    payment = call_razorpay(order)
    with lock:

```

No lock needed — no shared state
 # No lock needed — independent API call
 # Lock ONLY the shared data access

```
        update_database(order)
    send_email(order)                # No lock needed – independent
```

In the bad version, if the Razorpay call takes 500ms, every other thread waits 500ms even though the API call has nothing to do with shared state. The good version only blocks other threads for the milliseconds needed to update the database.

9. When to Lock, When Not To

Lock When:

Multiple threads READ and WRITE the same shared data.

```
# NEED a lock: two threads modifying the same counter
counter = 0
def increment():
    global counter
    with lock:
        counter += 1 # READ + WRITE shared data
```

```
# NEED a lock: two threads modifying the same list
shared_results = []
def add_result(result):
    with lock:
        shared_results.append(result) # Modifying shared list
```

Don't Lock When:

Threads only READ (no modification):

```
# NO lock needed: all threads only read the config
config = {"api_key": "abc123", "timeout": 30}

def fetch_data(url):
    timeout = config["timeout"] # READ only – safe without lock
    return requests.get(url, timeout=timeout)
```

Threads work on independent data:

```
# NO lock needed: each thread has its own data
def process_image(image_path):
    img = load_image(image_path) # Local variable – not shared
    resized = resize(img)        # Local variable – not shared
    save(resized, image_path + ".thumb")
    return image_path
```

Using thread-safe data structures:

```
import queue

# NO lock needed: queue.Queue is already thread-safe
task_queue = queue.Queue()
```

```
task_queue.put("task1")      # Thread-safe
task = task_queue.get()      # Thread-safe
```

The Balance

Too few locks:	Too many locks:
Race conditions, corrupted data, impossible bugs.	Everything runs sequentially, no concurrency benefit.

Find the minimum locking that ensures correctness.

Question: Five threads each read from a shared dictionary but never modify it. Do you need a lock?

► Answer

10. Connecting to What's Next

We've covered the **simplest** lock — a Mutex (allow exactly 1 thread at a time). But what if you want to allow **N** threads at a time? And what happens when locks go wrong?

Today (Concurrency-2.5)	Next: Concurrency-3
Executor syntax: <code>submit()</code> , <code>map()</code> , <code>as_completed()</code>	Semaphores (allow N threads, not just 1)
Parallel Merge Sort	Deadlocks (two threads each waiting for the other's lock)
Mutex (1 thread at a time)	Producer-Consumer pattern

Semaphore preview: A mutex is a lock that allows 1 thread. A semaphore allows **N** threads. Think of a parking garage with 5 spots — up to 5 cars can enter, the 6th waits. Useful for rate-limiting API calls (e.g., "only 5 concurrent requests to Razorpay").

Deadlock preview: What happens when Thread A holds Lock 1 and waits for Lock 2, while Thread B holds Lock 2 and waits for Lock 1? Both wait forever. Neither can proceed. This is a deadlock — and it's horrifying to debug in production.

11. Resources

- [Python docs — concurrent.futures](#) — Official documentation for executors and futures
- [Real Python — concurrent.futures](#) — Practical guide with examples
- [Real Python — Threading](#) — Covers locks and race conditions
- [Python docs — threading.Lock](#) — Official Lock documentation
- [Merge Sort Visualization](#) — Interactive sorting algorithm visualizer
- [David Beazley — Python Concurrency From the Ground Up \(YouTube\)](#) — Deep dive on locks and threading